Feedback for W10

Student: Yihao Yu (35567082)

IMPORTANT NOTICES

- The questions/responses' formatting may be broken in this file. Please refer to Moodle.
- If you disagree with any of your marking, please make a private post on Ed.
- If you need more help understanding why your marker didn't accept any of the answers, or would like more feedback, please book a consultation session.

FINAL OUTCOME: COMPETENT

Breakdown

- **Competent: 5**
- **Mistakes: 1**
- **Misunderstanding of Topics: 0**

Section 1

Status: COMPETENT

Question 1

Recall the implementation of Hash Tables with separate chaining:

> View Code

Suppose we have a Hash Table with separate chaining, with a table_size of 10, and we insert the following values:

* Mark, with hash("Mark") returning 7
* Alice, with hash("Alice") returning 5
* Jerry, with hash("Jerry") returning 5
* Talula, with hash("Talula") returning 3
* Kim, with hash("Kim") returning 7

How many SPNode objects will have their next attribute not equal to nullptr?

Response:

2

No feedback provided.

Section 2

Status: COMPETENT

Question 2

Drag the correct answers into the correct locations.

A linked list of size [n] has worst-case complexity .... for contains and .... for retrieving the 4th element. (The contains method tests whether a value is contained within the list) ;

Response:

{[O(n)]} {[O(1)]}

No feedback provided.

Section 3

Status: COMPETENT

Question 3

Inside the code scaffold below, implement the remainder of the BST method _search_helper, which receives a current node in the tree, and a search key, and tries to find the associated value within the tree. We've already provided the two base cases (no node, or key is found). YOU MUST IMPLEMENT THE RECURSIVE STEPS!

Don't worry about your code's highlighting and indentation or anything. Just write the code and the logic of the program. We understand this is not exactly a code editor, so it's fine if the code doesn't look great. It just needs to have the correct logic.

Response:

1

```
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
```

```
struct Node {
int key;
Node* left = nullptr;
Node* right = nullptr;
};

Node* _search_helper(Node* current_node, int key) {
// If we are outside the tree, that means the key can't be found
if (current_node == nullptr) return nullptr;
// If current_node node has the key, return it
if (current_node->key == key) return current_node;

// Otherwise, check which child we should go to.
// Fill me in!

if (current_node->key < key){

return _search_helper(current_node->right, key);

}

return _search_helper(current_node->left, key);



}
```

No feedback provided.

Section 4

Status: COMPETENT

Question 4

Suppose we have a MaxHeap mh with mh.array = [0, 9, 4, 8, 2, 3, 6, 1] and mh.size = 7. Drag the elements below into their correct current heap position. If an element in the array is not used in the heap representation, that is ok - just don't drag it onto the heap!

Response:

Drop zone 1 -> {2. 9} Drop zone 2 -> {3. 4} Drop zone 3 -> {4. 8} Drop zone 4 -> {5. 2} Drop zone 5 -> {6. 3} Drop zone 6 -> {7. 6} Drop zone 7 -> {8. 1}

Question 5

Your friend has tried to add the heapify implementation to the MaxHeap above, which currently has a bug in it:

```
// Construct this heap with the given array input
void heapify(int arr[], int n) {
    // First of all, create a new array for the heap with enough space - we need n+1 spots.
    delete[] array;
    array = new int[n + 1];
    current_capacity = n;

    // Copy all items into the array
    for (int i = 0; i < n; i++) {
        array[i] = arr[i];
    }
    size = n;

    // Go backwards through the nodes, and sink them down as far as they can go.
    for (int i = n; i >= 1; i--) {
        sink(i);
    }
}
```

Your friend knows there is a bug, because when she tried to heapify [1, 2, 3, 4], when calling extract_max on the resulting heap she received 4, 3, 2 and -5878654. The correct order, however the 1 seems to have been replaced with this other junk.

This is particularly weird, because when you look at the array stored in the heap before extracting, the 1 is initially present: [1, 4, 3, 2, -5878654].
Explain what the bug in the code is, and why this causes the 1 to be missing from output.

DON'T WRITE MORE THAN 40 WORDS. Manage your time; the question doesn't need a long answer.

Response:

The bug in the code is that it should be array[i+1] = arr[i] in the first for loop, since the first value in the heap structure locates in index 1 rather than index 0. This causes the 1 to be missing from output because the extract_max return the value on the array[1], and 1 do not sink due to its index so that 1 is totally excluded by the heap.

Question 6

Heap Sort, as you'll recall, is implemented by heapifying an array [1], then repeatedly extracting the maximum from this heap [2].

State the worst case complexity of part [1] and part [2] separately, and then the final combined complexity as a simplified tight Big-O bound.

Response:

The worst case complexity of part [1] is O (n).

The worst case complexity of part[2] is O (n log n).

The final combined complexity is O (n log n).

No feedback provided.

Section 5

Status: MISTAKES

You've been given a sorted list of values and want to analyse the statistics of the outliers of your data. In particular you want to know, for some number k, the total of the k smallest and largest values in the data.

You've cooked up the following code to achieve your result:

```
int total_outliers(int* sorted_arr, int n, int k) {
   // Finds the sum of the k outliers in an array of n sorted elements.
   Queue q(n);
   Stack s(n);
   for (int i=0; i<n; i++) {
      q.enqueue(sorted_arr[i]);
      s.push(sorted_arr[i]);
   }
   int total = 0;
   for (int i=0; i<k; i++) {
      // Invariant here!
      total = total + q.dequeue() + s.pop();
   }
}
```

Question 7

Suppose our sorted_arr is [1, 3, 7, 10, 12, 14, 15, 18]. On iteration 3 (i=2) of the second loop, the stack will pop {1; 3; 7; 10; 12; 14; 15; 18} and the queue will dequeue {1; 3; 7; 10; 12; 14; 15; 18}.

Response:

part 1: 14; part 2: 7

Question 8

What is the worst-case time complexity of the function?

Explain your answer with reference to the complexities of stack/queue operations.

DON'T WRITE MORE THAN 60 WORDS HERE. Just state the complexity, define variables, and explain it a bit.

Response:

The worst-case time complexity of the function should be O(n + k), the n is the size of the array, while k is the numer input to get the sum of k th smallest and k th largest values.

Question 9

Provide an invariant that holds true at the comment position (line 11) that would allow us to prove that the function will return the sum of the k smallest and k largest values in the data. Be sure to include everything that is known about the stack and the queue, and the current value of total.

Remember to word your invariant in terms of existing variables within the code, and also to ensure your invariant would work for the maintenance step (If we only know something about total, then our maintenance would likely fail).

DON'T WRITE MORE THAN 60 WORDS. You're not supposed to write the full proof, only the invariant statement. Manage your time; don't spend too long on any one question.

Response:

`The total is the sum of arry[0:i] and array[n-i:n].`

Feedback:

- Doesn't demonstrate a good understanding of invariant proofs
- The invariant statement doesn't capture the state of all necessary variables (total value, all items in the stack and the queue and their order)

Section 6

Status: COMPETENT

Suppose we've taken the standard merge sort algorithm, except we've replaced one line in the merge approach.

> View Code

But suppose we changed line 52:

`if (left_half[left_half_index] <= right_half[right_half_index]) {`

to instead just do:

if (rand_bool()) {

where rand_bool returns true 50% of the time and false 50% of the time.

Question 10

If we called merge sort after this edit, would it correctly sort the list?

If yes, explain why. If no, give a small example to support your argument.

DON'T WRITE MORE THAN 70 WORDS IN TOTAL. Manage your time; the question doesn't need a long answer.

Response:

It would not correctly sort the list.

For example:

left_half: [1,4,5]

right_half: [2,3,7]

if the rand_bool() return false, the merge array of these two array would append 2 at index 0, but the demanded result should be 1. In this case, the merge array could no longer be sorted anymore. And since it's not sort array, it would affect the execution of other recursion.

Question 11

Suppose we've added a level parameter to merge sort like so:

```
void merge_sort(int arr[], int n, int depth) {
  // ...
  merge_sort(left_half, left_half_size, depth + 1);
  merge_sort(right_half, right_half_size, depth + 1);
  // ...
}
```

so that we can track the recursive depth.

What is the largest depth will be throughout execution if we called merge_sort(arr, 32, 0) (So for the first call, level = 0), and why?

Response:

The largest depth would be 5. Since in the merge sort, if (n <= 1), the method would return. Therefore, log2 32 = 5 and in the depth 5, it would simply return itself as an array with size 1.

Question 12

Explain, in a good level of detail, the complexity of merge_sort. Your answer should reference your response to the previous question.

DON'T WRITE MORE THAN 100 WORDS HERE. Just show us that you know how the time complexity is calculated for a recursive function like this.

Response:

The complexity of merge_sort is O(n log n). That is because we have log2 n depth, and in each depth, we need to compare O(n) times and append n elements. Above all, we could simply multiply log2 n with O(n), hence the complexity of merge_sort should be O (n log n).

No feedback provided.

Marked by: william.wei, palash.rambhia, phoebe.jiang, ali.toosi, bardi.harborow